

CS103: Database Management Systems — Transactions & Storage

Department of Computer Science & Engineering | Academic Handout

Page 1: Relational Schema Normalization & ACID Guarantees

3.1 Relational Database Normalization

Normalization decomposes relations to minimize data redundancy and prevent insertion, update, and deletion anomalies:

- First Normal Form (1NF): All attributes contain atomic, indivisible values, and every tuple in the relation has a unique identifier (primary key).
- Second Normal Form (2NF): Relation is in 1NF and contains no partial functional dependencies (no non-prime attribute is functionally dependent on a proper subset of any candidate key).
- Third Normal Form (3NF): Relation is in 2NF and contains no transitive dependencies (for every functional dependency $X \rightarrow Y$, either X is a superkey or Y is a prime attribute).
- Boyce-Codd Normal Form (BCNF): A stricter variant of 3NF. A relation is in BCNF if and only if for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a candidate superkey.

3.2 ACID Transactional Properties

A database transaction represents a logical unit of work that must satisfy the ACID criteria:

- Atomicity: All modifications performed by the transaction are successfully committed to the database, or the transaction is entirely aborted and rolled back ('all-or-nothing'). Enforced via Write-Ahead Logging (WAL) and undo logs.
- Consistency: A transaction transitions the database from one valid state to another valid state, preserving all integrity constraints, foreign keys, and business rules.
- Isolation: The execution of concurrent transactions produces the same state as if the transactions were executed sequentially without interference. Isolation prevents intermediate uncommitted changes from being visible to external transactions.
- Durability: Once a transaction commits, its effects persist permanently in non-volatile storage, even in the event of subsequent system crashes, power failures, or OS crashes. Enforced via redo logs.

Page 2: Concurrency Control, Isolation Levels & Transaction Anomalies

3.3 Transaction Concurrency Anomalies

Concurrent execution without proper isolation leads to classical data anomalies:

- Dirty Read: Transaction T1 updates a row without committing; Transaction T2 reads the uncommitted value; T1 aborts, leaving T2 with invalid phantom data.
- Non-Repeatable Read (Fuzzy Read): Transaction T1 reads a row; Transaction T2 modifies or deletes that row and commits; T1 re-reads the row and discovers that the values have changed.
- Phantom Read: Transaction T1 reads a set of rows satisfying a search predicate; Transaction T2 inserts or deletes rows matching that predicate and commits; T1 re-evaluates the query and discovers a differing set of rows.

3.4 ANSI SQL Isolation Levels

Standard SQL defines four transaction isolation tiers offering trade-offs between concurrency and correctness:

1. Read Uncommitted: Allows Dirty Reads, Non-Repeatable Reads, and Phantom Reads. Lowest overhead.
2. Read Committed: Prevents Dirty Reads. Guarantees that any data read is committed at the moment of reading. Allows Non-Repeatable Reads and Phantom Reads (Default in PostgreSQL, Oracle, SQL Server).
3. Repeatable Read: Prevents Dirty Reads and Non-Repeatable Reads. Guarantees that rows read remain unchanged throughout the transaction. Phantom Reads may still occur (Default in MySQL InnoDB via Next-Key Locking).
4. Serializable: The strictest isolation level. Prevents all anomalies including Phantom Reads by simulating sequential execution via Two-Phase Locking (2PL) or Serializable Snapshot Isolation (SSI).

Page 3: Storage Engines: B+ Trees, LSM-Trees & Hash Indexes

3.5 B-Tree and B+ Tree Index Structures

B+ Trees represent the ubiquitous standard index structure for relational storage engines (e.g. InnoDB, SQLite).

A B+ Tree is a self-balancing, m-way search tree where:

- All actual data records (or pointers to data tuples) reside exclusively in the leaf nodes.
- Leaf nodes are doubly linked sequentially, enabling highly efficient range queries (e.g., `SELECT * WHERE id BETWEEN 100 AND 500`) through sequential scanning.
- Internal nodes store solely routing keys and child pointers, providing high fan-out and shallow tree height (typically 3 to 4 levels for millions of records), requiring minimal disk I/O.

Search, insertion, and deletion complexity is guaranteed $O(\log n)$.

3.6 Log-Structured Merge-Trees (LSM-Trees) vs Hash Indexes

LSM-Trees are optimized for write-heavy database workloads (e.g. Cassandra, RocksDB). Writes are appended sequentially to an in-memory MemTable backed by a Write-Ahead Log (WAL). When full, the MemTable is flushed to disk as an immutable SSTable (Sorted String Table). Background compaction merges overlapping SSTables. LSM-Trees provide exceptional write throughput at the expense of higher read amplification.

Hash Indexes map keys to fixed slots using a cryptographic or non-cryptographic hash function. Hash indexes provide $O(1)$ average-case point lookups (equality comparisons like `WHERE id = 42`), but cannot support range queries or partial prefix matches because hashes do not preserve lexical ordering.